

Author verification using Support Vector Machine

Introduction

Author verification (Same-author detection) is the task of determining whether two articles are written by the same author. Unlike traditional text classification, this problem focuses on analyzing the writing style rather than the topic or the meaning of the articles.

In this problem, Support Vector Machine (SVM) was used to classify whether pairs of articles are written by the same author. The objective is to evaluate the effectiveness of SVM for this problem and analyze the impact of different hyperparameter settings on model performance.

The workflow is divided into three phases:

- Data Preparation: Transforming raw data into article pairs.
- Hyperparameter Optimization: Executing a custom grid search with internal cross-validation over multiple seeds to identify optimal hyperparameter.
- Final Evaluation: Assessing final model's performance across multiple seeds.

SVM

1. Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm widely used for classification tasks due to its strong generalization capability and effectiveness in high-dimensional feature spaces. The primary objective of SVM is to identify an optimal decision boundary, known as a hyperplane, that maximizes the margin.

For non-linearly separable data, SVM uses kernel functions to project the original feature space into a higher-dimensional space where the discrimination becomes easier. In this project, the Radial Basis Function (RBF) kernel was utilized because of its ability to model complex non-linear relationships between stylometric features.

2. Radial Basis Function (RBF)

The Radial Basis Function (RBF) kernel is one of the most widely used kernel functions in Support Vector Machine models due to its ability to handle complex non-linear relationships. The RBF kernel is defined mathematically as:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Where x_i, x_j are two feature vectors and

$\|x_i - x_j\|^2$ represents the squared Euclidean distance between the two samples,

The RBF kernel computes the similarity between two feature vectors based on their distance in the feature space. When two samples are close together, the Euclidean distance becomes small, producing a kernel value near 1. Conversely, when the samples are far apart, the kernel value approaches 0.

The performance of the non-linear SVM is heavily dependent on two critical hyperparameters:

- **C:** Controls the regularization between maximizing the decision boundary margin and minimizing training errors. A lower C encourages a smoother, more generalized boundary, while a higher C penalizes misclassifications heavily, potentially leading to overfitting.
- **gamma:** Controls how far the influence of a sample extends in the feature space. In other words, it scales the value of the distance between two samples. Based on the formula:
 - If gamma is large, the similarity is diminished compared to the distance. Only samples located extremely close together are considered similar (near influence). This causes the model to create highly detailed and localized decision boundaries, increasing the risk of overfitting.
 - If gamma is small, the similarity is exaggerated compared to the distance. As a result, even relatively far samples still exert

influence on another one. This produces smoother decision boundaries with broader influence regions, improving generalization but potentially causing underfitting.

Data Preparation

1. Data Source

The dataset was collected from LessWrong forum ([lesswrong.com](https://www.lesswrong.com)). It includes 1484 articles written by 35 distinct authors. The distribution of articles per author is showed in the following table:

Author	Number of Articles	Author	Number of Articles
david-gross	50	jasoncrawford	44
screwtape	49	raemon	43
scottalexander	49	buck	42
sarahconstantin	49	joe-carlsmith	42
steve2152	48	ruby	42
adamshimi	48	zack_m_davis	41
kaj_sotala	48	so8res	40
ricraz	47	petermccluskey	40
zvi	47	jacob-falkovich	39
jessica-liu-taylor	47	holdenkarnofsky	38
katjagrace	47	lukeprog	38
tsvibt	47	elizabeth-1	36
benquo	46	benito	34
eliezer_yudkowsky	46	turntrout	33
johnswentworth	46	philh	29
gordon-seidoh-worley	46	jan_kulveit	27
abramdemski	45	nunosempere	27
ryan_greenblatt	44		

2. Feature Extraction

.....

3. Data splitting

If article pairs are randomly divided into training and test sets, articles belonging to the same author may appear in both sets. Therefore, the model will suffer from data leakage, causing it to memorize writing styles rather than generalize.

To prevent from this problem, the implementation performs an author-based split rather than an article-based split. Firstly, we extract the set of all unique authors in the dataset. The array of unique authors then was divided such that 80% of the authors (28 authors) were assigned to the training set, and the remaining 20% (7 authors) were reserved for the test set. Consequently, the authors present in the test set were entirely disjoint from those in the training set.

4. Article-Pairs Generation

4.1. Features

Because we consider author verification as a binary classification problem, the original dataset was transformed into a pairwise format. Consider two articles 1 and 2, let f_1 , f_2 be the feature vectors extracted from article 1 and article 2 respectively. To provide the SVM a comprehensive representation of stylistic similarity and divergence, the final combined feature vector was concatenated using three distinct operations:

- **Absolute Difference:** $|f_1 - f_2|$

It linearly captures the magnitude of stylistic differentiation between the two texts, highlighting features where the authors' writing habits diverge. If two documents are written by the same author, we expect that many stylistic features should be similar.

- **Squared Difference:** $(f_1 - f_2)^2$

Squared difference is similar to absolute difference, but it emphasizes large deviations more strongly. Some features may be highly

discriminative. By amplifying mismatches between these features, the model can pay more attention to features that better distinguish between authors.

- **Element-wise Multiplication:** $f1 * f2$

The element-wise multiplication can capture the similarity of specific writing habits. While 2 above distance metrics cannot distinguish between "both articles never use a feature" and "both articles use it frequently" (both differences are zero), multiplication produces a remarkably high value only when both texts strongly exhibit the same trait, providing the model with a strong indicator of identical authorship.

4.2. Generation

The resulting pairwise dataset was strictly balanced, containing exactly 50% positive pairs (labeled as 1) and 50% negative pairs (labeled as 0).

Positive pairs were generated by taking combinations of articles written by the same author. To prevent authors with large number of articles from dominating the training distribution, a maximum threshold of 200 positive pairs per author was enforced. If an author's combinations exceed this limit, we utilize uniform random sampling to extract exactly 200 pairs.

To achieve perfect class balance, the pipeline counts the total number of generated positive pairs. It then generates negative pairs with the exact number of positive pairs through a randomized sampling process. Firstly, two distinct authors are uniformly sampled from the author pool. Then, one document index is randomly selected for each of the two authors to make a pair.

Hyperparameter Optimization

2. Pipeline

In traditional machine learning workflows, hyperparameter tuning is typically executed using automated modules like Scikit-Learn's GridSearchCV. However, applying standard grid search methods to pairwise authorship verification leads to data leakage. Because standard cross-validation splits data at

the row level (based on pair), articles belonging to the same author may appear in both training and validation sets. This is the same problem with the one when we did data splitting. Therefore, we can solve this by using Grid-Search based on authors instead of articles.

To discover the optimal C, gamma configuration and ensure the model's generalizability, a three-step experimental pipeline was implemented:

- Author-Based K-Fold Cross-Validation

The implementation performs a 5-fold cross validation strategy in which the splitting process is performed at the author level rather than at the articles or pair level. It means that articles written by the same author never appear simultaneously in both the training folds and the validation fold.

More specifically, all unique authors are first divided into five mutually exclusive folds. Then in each iteration, 4 folds (80% of the authors) were used for training set, while the remaining 1 fold (20% of the authors) served as the validation set. This process is repeated five times so that each fold serves as the validation set exactly once.

- Fold-Wise Pair Generation and Feature Scaling

A critical aspect of the implementation is that pair generation is performed inside the cross-validation loop rather than before dataset splitting. For every fold, pair generation is performed separately for: training authors and validation authors.

After pair generation, feature scaling is applied using StandardScaler:

$$Z = (x - \mu) / \sigma$$

where:

μ is the mean,

σ is the standard deviation

The scaler is fitted only on the training set and then applied to validation set.

- Grid Search

Hyperparameter tuning was performed using an exhaustive grid search over multiple combinations of the SVM parameters C and γ . The search space for C is $[0.1, 1, 10]$ and gamma is $[0.001, 0.01, 0.1]$. This produces a total of: $3 \times 3 = 9$ distinct hyperparameter configurations.

For each combination of hyperparameters, the algorithm records the training and validation accuracies across all 5 folds. To further improve the robustness and reliability of the experimental results, the entire pipeline was repeated 5 times using 5 different random seeds (19, 20, 21, 22, 23). The final configuration was chosen manually based on the accuracies on both training and validation sets.

3. Grid-Search Result

The aggregated results, including Mean Training Accuracy, Mean Validation Accuracy, and the Generalization Gap (Train - Val), are summarized in Table 1.

C	gamma	Mean Train Acc	Mean Val Acc	Gap	observation
0.1	0.001	0.7657 $\pm$ 0.0047	0.7284 $\pm$ 0.0050	0.0373	Most Stable (Lowest Gap)
0.1	0.01	0.6909 $\pm$ 0.0019	0.6105 $\pm$ 0.0081	0.0804	Underfitting
0.1	0.1	1.0000 $\pm$ 0.0000	0.5000 $\pm$ 0.0000	0.5	Overfitting
1	0.001	0.8398 $\pm$ 0.0042	0.7327 $\pm$ 0.0072	0.107	Highest Validation Accuracy
1	0.01	0.9954 $\pm$ 0.0005	0.6942 $\pm$ 0.0073	0.3012	Overfitting
1	0.1	1.0000 $\pm$ 0.0000	0.5001 $\pm$ 0.0003	0.4999	Overfitting
10	0.001	0.9585 $\pm$ 0.0023	0.7204 $\pm$ 0.0052	0.238	Overfitting
10	0.01	1.0000 $\pm$ 0.0000	0.6907 $\pm$ 0.0061	0.3093	Overfitting
10	0.1	1.0000 $\pm$ 0.0000	0.5002 $\pm$ 0.0004	0.4998	Overfitting

Table 1: SVM Hyperparameter Tuning Results (Aggregated across 5 Seeds)

(Note: Detailed fold-by-fold results for all 5 seeds can be found in **Appendix A**).

First, models using a large gamma value ($\gamma=0.1$) consistently achieved perfect training accuracy (100%) across all seeds, while the validation accuracy remained at approximately 0.5. This behavior indicates severe overfitting, where the model memorizes the training data but fails to generalize to unseen authors. A validation accuracy close to 0.5 suggests that the classifier performs nearly at random guessing on the validation set.

Similarly, configurations with large C values, especially $C=10$, also produced extremely high training accuracy. However, their validation performance were not high correspondingly.

Excluding the overfitted and underfitted models, the decision for the final hyperparameters narrowed down to the two most dominant configurations:

- Configuration A ($C=1$, $\gamma=0.001$): Achieved the highest absolute Mean Validation Accuracy (73.27%) but exhibited a noticeable generalization gap between training and validation ($\sim 10.7\%$).
- Configuration B ($C=0.1$, $\gamma=0.001$): Achieved a slightly lower Mean Validation Accuracy (72.84%) but demonstrated a tight generalization gap ($\sim 3.73\%$).

The configuration with $C=0.1$ and $\gamma=0.001$ was selected for the final evaluation phase. Although the configuration $C=1$ achieved a slightly higher validation accuracy, the model with $C=0.1$ demonstrated substantially better generalization ability. Specifically, the generalization gap was only approximately 3.73%, compared to 10.7% for $C=1$. This smaller gap indicates that the model trained with $C=0.1$ is more stable and less sensitive to author-specific noise or overfitting. Therefore, sacrificing roughly 0.4% validation accuracy is considered a worthwhile trade-off in exchange for significantly improved robustness and generalization performance on unseen authors.

Final Model Evaluation

Following the hyperparameter optimization stage, the final Support Vector Machine (SVM) model was retrained using the complete training set. Unlike the

cross-validation phase, where the model was trained on only subsets of the training authors, the final model utilized all authors assigned to the training set.

Specifically, pairs were generated using all unique authors designated to the training set. A final instance of StandardScaler was fitted exclusively on this comprehensive training set. The SVM was then fitted on this fully scaled dataset.

It is important to emphasize that the test set remained completely disjoint from the training data throughout the entire experimental process. All test pairs were constructed exclusively from authors that were never observed during training, cross-validation, or hyperparameter tuning. Consequently, the evaluation measures the model’s true ability to generalize to previously unseen writing styles rather than memorizing author-specific patterns. The model’s performance was evaluated using multiple classification metrics across 5 seeds (Seeds 19 to 23), each containing 11200 training pairs and 2800 test pairs. The final performance was reported using multiple classification metrics, including Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC.

Seed	Accuracy	Macro Precision	Macro Recall	Macro F1-Score	ROC-AUC
19	0.7439	0.74	0.74	0.74	0.8234
20	0.735	0.74	0.73	0.73	0.8164
21	0.6989	0.7	0.7	0.7	0.7823
22	0.7393	0.75	0.74	0.74	0.8386
23	0.7461	0.75	0.75	0.74	0.8086
Mean ± Std	0.7326 ± 0.0189	0.7360 ± 0.0207	0.7320 ± 0.0205	0.7300 ± 0.0187	0.8139 ± 0.0186

Table 2: Final Evaluation Metrics on Test Set (Over 5 Seeds)

1. Performance

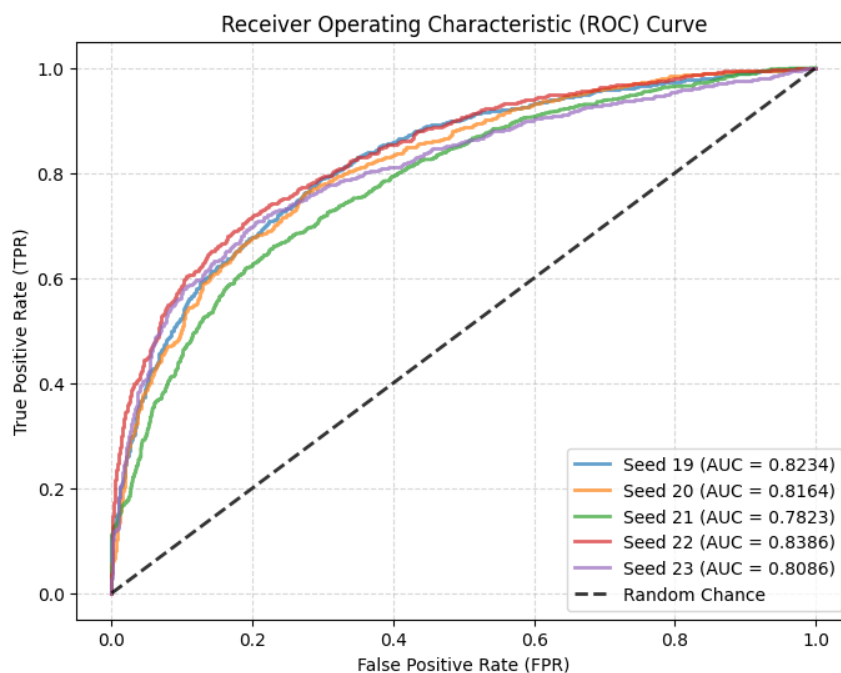
The most prominent takeaway from the results is the low standard deviation across all evaluated metrics. The Mean Accuracy stands at 0.7326 with a fluctuation of ± 0.0189 (1.89%). Similarly, the Macro F1-score remains highly stable at 0.7300 ± 0.0187 . This minimal variance demonstrates that the model is highly general. This means that despite being exposed to completely different, unseen authors in each of the 5 seeds, the model's performance fluctuates by nearly 2%. As a result, model’s performance is not a result of a lucky data split. The

model has successfully captured stylistic patterns that generalize well to entirely unseen authors.

However, some variations between seeds can still be observed. For example, Seed 21 achieved the lowest overall accuracy and ROC-AUC score, suggesting that this particular author split may contain more challenging writing patterns or stylistically similar authors.

The table also shows a perfect symmetry: Mean Accuracy (0.7326), Macro Precision (0.7360), Macro Recall (0.7320), and Macro F1-score (0.7300) are all tightly clustered around the 73% mark. This suggests that the classifier does not exhibit a strong bias toward either the positive or negative class.

2. Receiver Operating Characteristic (ROC) Analysis



To further evaluate the model's discriminative ability on different threshold, the Receiver Operating Characteristic (ROC) curves were plotted for all 5 seeds.

The red curve representing Seed 22 achieves the highest ROC-AUC score (0.8386), suggesting superior discrimination between the two classes for that seed. In contrast, Seed 21 obtained the lowest ROC-AUC score (0.7823), reflecting a more challenging stylistic distribution within its testing authors.

Nevertheless, the average ROC-AUC score of 0.8139 confirms that the classifier performs substantially better than random guessing. Overall, the ROC analysis validates that the model captured stylistic relationships between articles, leading to a strong performance even on previously unseen authors.

3. Confusion matrix

To understand more about the model’s predictive behavior beyond aggregated metrics, 5 confusion matrices over 5 seeds and the average matrix were given:

Seed 19		Seed 20		Seed 21	
1017	383	1045	355	901	499
334	1066	387	1013	344	1056
Seed 22		Seed 23		Average	
909	491	1142	258	1002.8	397.2
239	1161	453	947	351.4	1048.6

Table 3. Confusion matrices over 5 seeds

Actual \ Predicted	Different (0)	Same (1)
Different (0)	TN	FP
Same (1)	FN	TP

The average confusion matrices across the five experimental runs reveal that the SVM classifier achieves relatively balanced predictions between the “Same Author” and “Different Author” classes. In most cases, the numbers of true positives and true negatives remain reasonably high, indicating that the model can effectively distinguish stylistic similarities and differences between document pairs.

The model registered an average of 397 False Positives per run. False Positives typically occur when two distinct authors share a same academic

background, or strictly adhere to formal structural templates. This shared contextual vocabulary and syntactic rigidity cause their interaction vectors to artificially spike, tricking the SVM into perceiving a unified authorial signature.

In contrast, the model produced an average of 351 False Negatives. This error highlights the inherent fluidity of human language. A single author may drastically change writing style depending on the target audience, the emotional context of the text, or chronological evolution over time.

While the aggregated macro-metrics remain highly stable, a granular inspection of the individual confusion matrices reveals significant variance in class-wise Precision and Recall across different test seeds. For example, in Seed 22, the model exhibited a strong bias toward predicting identical authorship (Recall for Class 1 peaked at 0.83, while Class 0 dropped to 0.65). Conversely, Seed 23 demonstrated the exact opposite behavior (Recall for Class 0 reached 0.82, while Class 1 fell to 0.68).

Because each seed samples a completely disjoint subset of authors for testing, the model encounters distinct linguistic distributions in every iteration. These fluctuations are expected because each seed generates a completely different author-level train-test split. Consequently, the classifier encounters distinct stylistic distributions and varying levels of writing similarity in every experimental run.

This inter-seed variability justifies the use of multi-seed evaluation rather than relying on a single random split. Evaluating the model on only one seed could produce a skew perception about model's predictive ability. By averaging the results across five independent experimental runs, the evaluation becomes significantly more robust, providing a more reliable and unbiased estimation of the model's true generalization capability in real-world authorship verification scenarios.